

How I Used Lean for Agentic Delegation and the Language Frontier of Software Developers: A Model and Evidence from Claude Code on GitHub

Source translation, machine-checked proofs, a boundary counterexample, and an honest partial-formalization verdict

Gabriel Saco

September 2, 2026

Universidad del Pacífico

Formalization of Quispe & Xu (2026), arXiv:2605.25438v2

github.com/gsaco/ai-03-quispe/tree/branch1

What I asked Lean to do

Mathematical question

Do the five named propositions follow from their stated threshold, ordering, finiteness, and monotonicity conditions?

Translation question

Does the Lean statement preserve the paper's domain, quantifiers, endpoints, inequalities, and conclusion?

Stress test

Are any strict claims false at an admissible boundary, or true only after adding a premise that the source does not state?

Not the question

Lean cannot establish that Claude adoption is exogenous, that GitHub measurement is correct, or that an event-study coefficient is causal merely by proving the model's algebra.

The target is a faithful implication: source assumptions in, source conclusion out.

Source pinning and formal scope

Pinned source

Paper: *Agentic Delegation and the Language Frontier of Software Developers: A Model and Evidence from Claude Code on GitHub*

Version: arXiv v2, July 7, 2026

SHA-256:

cddc048711c43022d5fd01b995bfb111
4c728c8c879b809b5fdc354a391d3c35

Inventory before proving

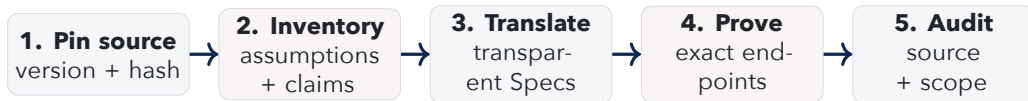
- ▶ three numbered assumptions;
- ▶ five numbered propositions;
- ▶ exact PDF locations;
- ▶ one source-facing Spec per proposition.

Deliberately out of scope

Empirical tables and figures, the event-study estimator, Bayesian learning, treatment detection, and causal identification.

I formalized the pinned source directly; the worked example was a workflow guide, not a code source.

The EconCSLib workflow I followed



Why statement first?

A proof of the wrong theorem is still a formalization failure. The paper-facing proposition is frozen before proof implementation.

Why audit after proof?

Compilation checks logic. The audit checks whether stronger assumptions, weaker conclusions, or changed domains entered silently.

Four layers keep meaning separate from proof engineering

PaperInterface.lean

Five transparent source-facing Spec : Prop declarations



ProofInterface.lean

Five exact-type proof endpoints



MainTheorems.lean

Definitions, helper lemmas, case splits, finite sums



Assumptions.lean

Only location allowed for local postulates; left empty

Human review surface

Read `PaperInterface.lean` first. It exposes the formal claim without proof tactics obscuring its meaning.

Audit surface

The JSON ledgers map source locations to Specs, classify scope gaps, and quarantine the Proposition 3 defect.

Visual dependency map

`docs/DependencyDAG.pdf` links the three assumptions to the propositions they support.

Formal theorem inventory

Source	Lean endpoint	Status and scope
Prop. 1	<code>frontierExpansion</code>	Proved: static menu and finite language count.
Prop. 2	<code>activationBandFor UnfamiliarLanguages</code>	Proved with an order-theoretic CDF abstraction.
Prop. 3	<code>dynamicCumulative LanguageEffectCorrected</code>	Corrected theorem proved; printed endpoint refuted.
Prop. 4	<code>specialistAndAbilityHeterogeneity</code>	Proved under explicit nonnegativity and monotonicity premises.
Prop. 5	<code>repositoryExpansion</code>	Proved pointwise for a finite repository type.

Logical result

All five exact-type endpoints compile without proof holes. Every economic restriction appears in the theorem signature or in a transparent source-facing specification.

Semantic result

Four targets preserve the finite core of the source claim. The fifth, Proposition 3, requires an explicit domain correction and therefore prevents a fully formalized label.

Proposition 1: adding a mode cannot shrink the frontier

1 • Paper mathematics

$$\max\{V^S, V^C, V^D\} \geq \max\{V^S, V^C\} \implies Z_{ikt}^2 \geq Z_{ikt}^1, \quad N_{it}^2 \geq N_{it}^1.$$

2 • Lean proof core

```
theorem frontierExpansionProof
  {Language : Type} [Fintype Language]
  (oldSurplus delegationSurplus : Language → Real) :
  (forall k, oldActive k ≤ newActive k) /\
    sum oldActive ≤ sum newActive := by
  have hpoint : forall k, oldActive k ≤ newActive k := by
    intro k
    by_cases hold : 0 ≤ oldSurplus k
    . exact hold.trans (le_max_left _ _)
  exact <hpoint, Finset.sum_le_sum ...>
```

3 • Interpretation

Languages form an arbitrary finite type. Lean first proves the indicator inequality for each language, then lifts it to the total count with `Finset.sum_le_sum`.

Exactness

This verifies the pathwise static menu result. It does not prove that delegation is adopted, strictly preferred, or empirically responsible for observed expansion.

Proposition 2: translating the activation band

1 • Paper equation

Under the no-foothold condition and $T^D < T^S$,

$$Z^2 - Z^1 = \mathbb{I}\{T^D \leq \omega < T^S\}, \quad \Pr(Z^2 - Z^1 = 1) = F(T^S) - F(T^D).$$

2 • Lean statement

```
theorem activationBandIndicator
  (TD TS opportunity : Real)
  (hthreshold : TD < TS) :
  (if TD <= opportunity then 1 else 0) -
    (if TS <= opportunity then 1 else 0) =
  if TD <= opportunity /\ opportunity < TS
  then 1 else 0 := by ...
```

3 • Object mapping

Thresholds and opportunities are real numbers. Activation is encoded by zero-one natural-number indicators. The half-open interval is represented by one weak and one strict inequality.

Exactness

The indicator identity is exact. The CDF statement is formalized as monotonicity of a real function, not as a constructed conditional probability measure.

The nontrivial step: rule out one impossible activation state

Case table

Delegation	Solo	Difference
inactive	inactive	zero
active	inactive	one
active	active	zero
inactive	active	impossible

Why impossible?

If the solo threshold is reached and the delegation threshold is strictly lower, then the delegation threshold must also be reached.

Lean fragment

```
by_cases hD : TD <= opportunity
. by_cases hS : TS <= opportunity
  . simp [hD, hS, not_lt_of_ge hS]
  . have hband : opportunity < TS :=
      lt_of_not_ge hS
    simp [hD, hS, hband]
. have hS : not (TS <= opportunity) := by
  intro hs
  exact hD ((le_of_lt hthreshold).trans hs)
simp [hD, hS]
```

What the tactic is doing

`by_cases` exhausts the economic states; `trans` applies threshold ordering; `simp` evaluates the zero-one indicators after each state is known.

Lean turns an intuitive threshold picture into an exhaustive proof with no missing state.

Proposition 3: Lean found a real endpoint failure

Printed strict claim

In the closed-frontier benchmark, define

$$g_s = 1 - (1 - p)^{s+1}.$$

The paper claims that g_s is strictly increasing with strictly diminishing increments whenever the post-agent hazard is positive.

Boundary substitution

At the admissible endpoint $p = 1$,

$$g_s = 1 - 0^{s+1} = 1$$

for every observed horizon. The effect jumps on impact and is flat afterward.

Formal consequence

Weak nonnegativity remains valid on the closed unit interval. Strict growth and strict discrete concavity require the additional upper bound $p < 1$.

This is a source-domain correction, not a technical convenience for Lean.

Proposition 3: corrected Lean theorem and proof step

1 • Corrected mathematical statement

For $0 < p < 1$, the cumulative gap grows strictly and its next increment is strictly smaller than its current increment.

2 • Lean statement and proof fragment

```
theorem closedFrontierStrictDynamics
  (horizon : Nat) (p : Real)
  (hpPositive : 0 < p) (hpBelowOne : p < 1) :
  let gap := fun h : Nat => 1-(1-p)^(h+1)
  gap horizon < gap (horizon+1) /\
    gap (horizon+2)-gap (horizon+1) <
    gap (horizon+1)-gap horizon := by
  have hpowDecrease :
    q^(horizon+2) < q^(horizon+1) := by
    rw [pow_succ]
    exact mul_lt_of_lt_one_right
      hpowPositive hqBelowOne
```

3 • Nontrivial line

Set $q = 1 - p$. Rewriting the later power as the earlier positive power multiplied by q reduces the comparison to the fact that multiplication by a number strictly between zero and one shrinks a positive quantity.

Exactness and counterexample

Lean verifies the corrected interior theorem exactly. `closedFrontierHazardOneNotStrict` separately proves that the paper's printed strict claim fails at the endpoint.

Propositions 4 and 5: aggregation without hidden probability

Proposition 4

A common activation increment is summed over a finite candidate type. Lean reduces the sum to candidate count times the common increment, then combines:

- ▶ nonnegative increments;
- ▶ monotonicity in ability;
- ▶ ordered candidate counts.

```
Finset.sum_const mul_le_mul
```

Proposition 5

For each repository, a weakly lower entry cost preserves every previously active indicator. A single strict activation-band witness makes the finite repository sum strictly larger.

```
Finset.sum_le_sum Finset.sum_lt_sum
```

Shared limitation

Both are pointwise finite aggregation theorems. Expectations inherit the inequality, but this formalization does not construct an explicit probability space.

Finite types make the counting logic explicit and keep probabilistic claims outside the proved boundary.

What is exact, abstracted, corrected, and excluded

Classification	Formalization disposition
Exact finite core	Menu inclusion, threshold indicator identity, finite sums, hazard ordering, and repository-count monotonicity.
Abstracted	Conditional CDF represented as a continuous monotone real function; no measure-theoretic conditional law is built.
Corrected	Proposition 3 strict dynamics restricted to an interior hazard; endpoint failure proved as a regression theorem.
Not formalized	GitHub data pipeline, staggered event study, standard errors, treatment exogeneity, skill measurement, Bayesian learning, tables, and figures.
Pending audit	Full v11 source-facing semantic closeout and human sign-off.

Why “partially formalized” is the right label

The proof bodies are complete. The remaining debt lies at the source-to-formal-statement boundary and in intentionally excluded empirical content—not in unresolved Lean goals.

Build and paper-scoped check

Full Lean target

```
LEAN_NUM_THREADS=1 lake build  
  QX26AgenticDelegation
```

Build completed successfully (8318 jobs).

Required EconCSLib check

```
python3 scripts/paper_contribution.py check  
  QX26AgenticDelegation --fast
```

Interface build passed; scoped diff check passed; exit code 0.

Proof hygiene

- ▶ no sorry;
- ▶ no admit;
- ▶ no local axioms;
- ▶ no native_decide;
- ▶ generated .gitignore respected.

Submission integrity

The exact generated paper folder was copied to `lean/`. Paper PDFs, extracted source bytes, caches, and private review traces remain ignored.

How to interpret the Lean result

Lean establishes

- ▶ five closed implication proofs;
- ▶ explicit domains and premises;
- ▶ exhaustive threshold cases;
- ▶ a valid correction to Proposition 3;
- ▶ a machine-checked endpoint refutation.

Lean does not establish

- ▶ that model assumptions hold empirically;
- ▶ that Claude caused observed language breadth;
- ▶ that the CDF abstraction is a full probability model;
- ▶ that the paper is fully formalized.

My verdict: the finite model core is mechanically sound, Proposition 3 needs a real domain repair, and the causal claim remains outside Lean.

Read next: `lean/FINAL_VALIDATION_REPORT.md` → `lean/PaperInterface.lean` → `lean/docs/DependencyDAG.pdf`.